

Revisão 1° GQ

Disciplina: Programação Orientada a Objetos

Aula: Revisão 01-13

Semestre: 2026.2

Prof.ª: Ivna Valença de Oliveira | ivna.oliveira@unicap.br

Bacharelado em Ciência da Computação

Universidade Católica de Pernambuco - UNICAP

Repositório da Disciplina

Todo o material de código da disciplina está disponível no GitHub. Acesse, clone e pratique!

🔗 Link do Repositório

github.com/poo-unicap/poo-java-2026.2

https://github.com/poo-unicap/poo-java-2026.2

📁 O que você encontra lá

- Exemplos de código de cada aula
- Exercícios e gabaritos
- Projetos práticos da disciplina
- Material de apoio e referências

⚙️ Como clonar o repositório

```
git clone https://github.com/poo-unicap/poo-java-2026.2.git
```

Mantenha seu fork atualizado a cada aula para ter acesso ao material mais recente.

O que vamos revisar hoje?

Quatro grandes blocos de conhecimento — do básico ao avançado da POO:

01 — Fundamentos de Java

Tipos de dados, variáveis, operadores, condicionais e repetição

02 — Métodos e Estruturas de Dados

Métodos, Strings, Arrays e ArrayList

03 — Classes e Objetos

Modelagem, atributos, métodos, construtores e encapsulamento

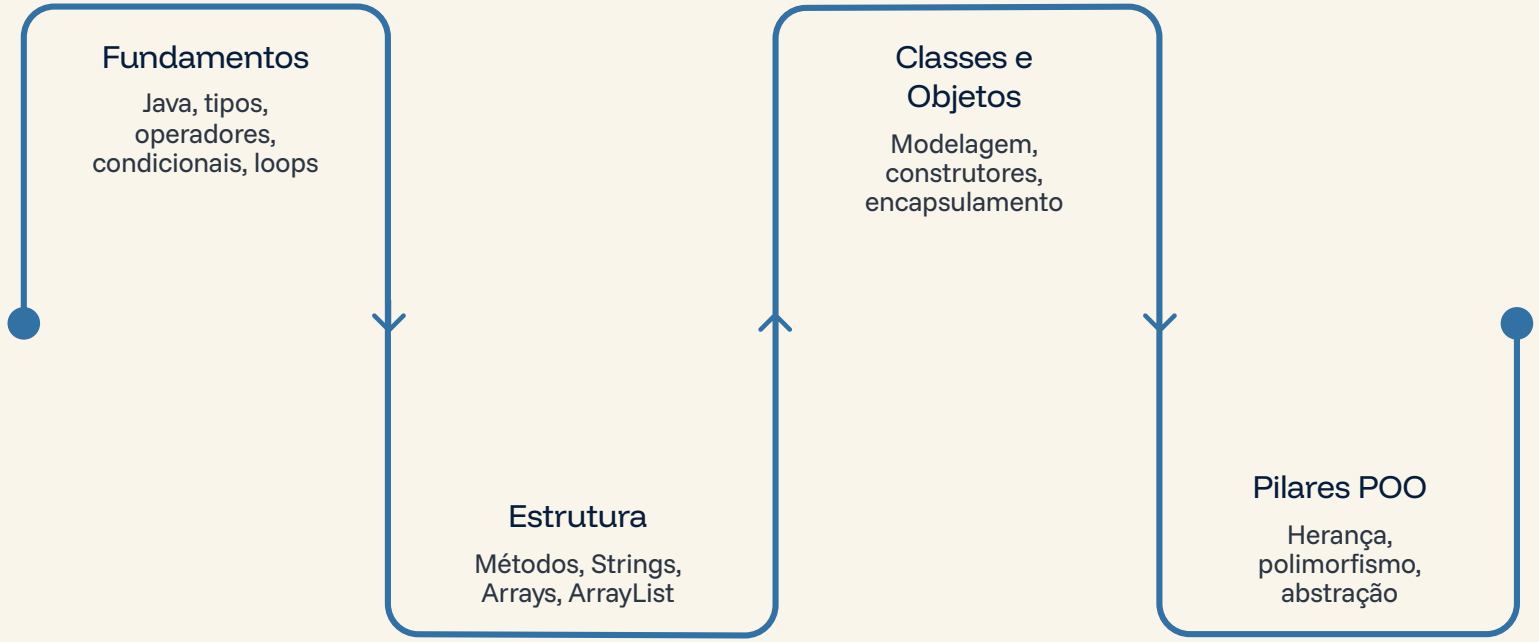
04 — Pilares da POO

Herança, polimorfismo e classes abstratas

🎯 A ideia central: CONCEITO → EXEMPLO → CÓDIGO → ANÁLISE → RACIOCÍNIO → APLICAÇÃO

Do Java à POO — Uma Evolução

Os conceitos das Aulas 01–13 não são capítulos isolados: cada um sustenta o próximo.



Você não aprende POO do zero — você já construiu essa base. Esta aula é para **organizar, conectar e praticar**.

Tipos de Dados e Variáveis

Tipos Primitivos

Tipo	Uso	Exemplo
`int`	Números inteiros	`int pedidos = 3;`
`double`	Números reais	`double saldo = 150.75;`
`boolean`	Verdadeiro/falso	`boolean ativo = true;`
`char`	Um caractere	`char status = 'A';`
`long`	Inteiro grande	`long cpf = 12345678900L;`

Tipo de Referência

```
// String: referência a objeto
String usuario = "maria_dev";
String codigo = "PED-20241001";

// Declaração + inicialização
int quantidade = 5;
double taxa = 0.0275;
boolean aprovado = false;
```

💡 `String` começa com maiúscula — é uma classe, não um tipo primitivo!

Operadores em Java

Aritméticos

```
saldo + deposito   // adição
preco * quantidade // multiplicação
total / parcelas   // divisão
pontos % 10        // resto
```

Relacionais

```
saldo >= 0      // maior ou igual
idade == 18     // igualdade
tentativas != 3 // diferente
nivel < 5       // menor que
```

Lógicos

```
ativo && saldo > 0  // E (ambos)
pix || ted          // OU (um dos dois)
!bloqueado          // NÃO (negação)
```

Atribuição

```
saldo = 100.0;  // atribuição
saldo += 50.0;  // saldo = saldo + 50
pontos *= 2;    // pontos = pontos * 2
nivel++;        // nivel = nivel + 1
```

⚠️ Atenção: `==` compara valores primitivos. Para comparar Strings, use `.equals()`!

Estruturas Condicionais

Estrutura

```
double saldo = 350.0;
double valor = 500.0;

if (saldo >= valor) {
    System.out.println("Transferência aprovada");
} else if (saldo > 0) {
    System.out.println("Saldo insuficiente");
    System.out.println("Saldo: R$ " + saldo);
} else {
    System.out.println("Conta bloqueada");
}
```

Raciocínio

1. Avalie a condição do `if`

   `350.0 >= 500.0` → **false**

2. Passe para o `else if`

   `350.0 > 0` → **true**

3. Execute o bloco verdadeiro

   Imprime "Saldo insuficiente" e o saldo

Estruturas de Repetição

Exemplo — quantas vezes executa?

```
int pontos = 0;
for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) {
        pontos += 10;
    }
}
System.out.println(pontos);
```

```
// while
int tentativas = 3;
while (tentativas > 0) {
    tentativas--;
}

// do-while — executa pelo menos uma vez
do {
    // processamento
} while (condicao);
```

Rastreando o for:

i	i%2==0?	pontos
1	false	0
2	true	10
3	false	10
4	true	20
5	false	20

✅ Saída: `20` — o loop executa 5 vezes, mas só soma quando i é par.

Métodos em Java

```java
// Método com retorno
public static double calcularTaxa(double valor,
                                  double percentual) {
    double taxa = valor * (percentual / 100);
    return taxa;
}

// Método sem retorno (void)
public static void exibirExtrato(String usuario,
                                 double saldo) {
    System.out.println("Usuário: " + usuario);
    System.out.println("Saldo: R$ " + saldo);
}

// Chamada no main
public static void main(String[] args) {
    double t = calcularTaxa(1000.0, 2.5);
    exibirExtrato("joao_dev", 850.0);
}
```

Anatomia de um Método

Modificador

`public static` — visibilidade e pertencimento

Tipo de Retorno

`double` → retorna valor · `void` → sem retorno

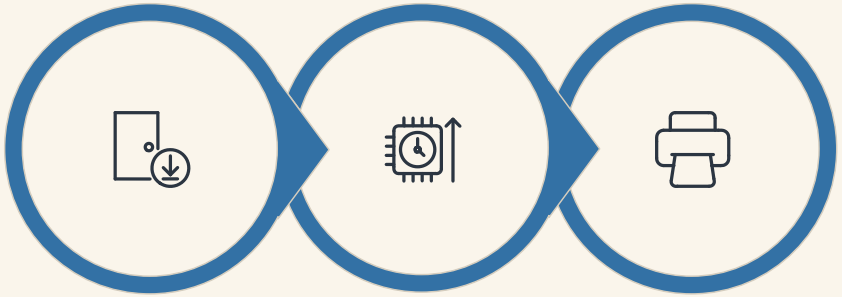
Parâmetros

Entradas que o método recebe para processar

Corpo

Lógica de processamento + `return`

Parâmetros e Retorno



Entrada | Processamento | Saída

Com retorno

```
public static int nivelUsuario(int xp) {
   return xp / 100;
}
// Chamada:
int nivel = nivelUsuario(350);
// nivel = 3
```

Sem retorno

```
public static void notificar(String msg) {
 System.out.println("[ALERTA] " + msg);
}
// Chamada:
notificar("Pedido entregue!");
```

Strings em Java

Métodos Essenciais

```java
String usuario = "maria_dev";
String pedido  = "PED-20241001";

// Comprimento
int tam = usuario.length();      // 9

// Comparação (nunca use ==!)
boolean igual = usuario.equals("maria_dev"); // true

// Maiúsculo / minúsculo
String up = pedido.toUpperCase(); // "PED-20241001"

// Substring
String cod = pedido.substring(0, 3); // "PED"

// Contém
boolean tem = pedido.contains("2024"); // true

// Concatenação
String msg = "Olá, " + usuario + "!";
String msg2 = "Olá, ".concat(usuario);
```

Ponto de Atenção

String não é tipo primitivo

É uma classe — objetos String têm métodos!

== compara referências

Use `.equals()` para comparar conteúdo de Strings

Índices começam em 0

`"PED-2024".substring(0,3)` → `"PED"`

Arrays em Java

```java
// Declaração e criação
int[] notas = new int[4];
String[] produtos = {"Teclado","Mouse","Monitor"};

// Acesso por índice (começa em 0)
notas[0] = 85;
notas[1] = 90;
notas[2] = 70;
notas[3] = 95;

// Percorrimento
for (int i = 0; i < notas.length; i++) {
   System.out.println(notas[i]);
}

// For-each (leitura)
for (String p : produtos) {
   System.out.println(p);
}

// Tamanho
int tamanho = notas.length; // 4
```

Visualizando o Array

Índice	0	1	2	3
Valor	85	90	70	95

Tamanho fixo

Definido na criação — não pode ser alterado

`.length` sem parênteses

É um atributo, não um método

Índice válido: 0 a length-1

Acesso fora causa `ArrayIndexOutOfBoundsException`

ArrayList em Java

```java
import java.util.ArrayList;

// Criação
ArrayList<String> playlist = new ArrayList<>();

// Adicionar elementos
playlist.add("Shape of You");
playlist.add("Blinding Lights");
playlist.add("As It Was");

// Acessar elemento
String musica = playlist.get(0); // "Shape of You"

// Tamanho
int total = playlist.size();    // 3

// Remover
playlist.remove("As It Was");

// Iterar
for (String m : playlist) {
    System.out.println(m);
}

// Verificar se contém
boolean tem = playlist.contains("Blinding Lights");
```

Array vs ArrayList

Característica	Array	ArrayList
Tamanho	Fixo	Dinâmico
Sintaxe	`int[]`	`ArrayList<T>`
Acesso	`[i]`	`.get(i)`
Tamanho	`.length`	`.size()`
Primitivos	Sim	Somente objetos

💡 Para tipos primitivos em ArrayList, use wrappers: `Integer`, `Double`, `Boolean`.

Modelagem de Classes

Do mundo real para o código

Um sistema de delivery precisa representar **pedidos**. Como modelamos isso?

Identifique a entidade

"Pedido" — o que caracteriza um pedido?

Defina os atributos

Código, cliente, valor total, status

Defina os comportamentos

Confirmar, cancelar, calcular total

Escreva a classe Java

Atributos + construtores + métodos

```
public class Pedido {

 // Atributos
 String codigo;
 String cliente;
 double valorTotal;
 String status;

 // Método
 public void confirmar() {
 this.status = "CONFIRMADO";
 System.out.println("Pedido " + codigo
 + " confirmado!");
 }

 public void cancelar() {
 this.status = "CANCELADO";
 }
}
```

Classe × Objeto



```
// Isso é a CLASSE (o modelo)
public class Pedido {
    String codigo;
    double valor;
}
```

```
// Isso cria OBJETOS (instâncias)
Pedido pedido1 = new Pedido();
pedido1.codigo = "PED-001";
pedido1.valor  = 89.90;

Pedido pedido2 = new Pedido();
pedido2.codigo = "PED-002";
pedido2.valor  = 150.00;
```

📌 Uma classe pode gerar infinitos objetos. Cada objeto tem seu próprio estado (valores dos atributos).

Atributos — O Estado do Objeto

Atributos = características/estado

```
public class ContaDigital {
    String titular;
    String numeroConta;
    double saldo;
    boolean ativa;
    int limiteMensalPix;
}
```

Cada objeto `ContaDigital` terá seus próprios valores para esses atributos.

Dois objetos, estados diferentes

Atributo	conta1	conta2
titular	"Ana"	"Bruno"
saldo	1250.00	340.50
ativa	true	false
limitePix	5000	2000

💡 O estado de um objeto é definido pelos valores armazenados em seus atributos a cada momento.

Métodos — O Comportamento do Objeto

```
public class ContaDigital {
    String titular;
    double saldo;
    boolean ativa;

    // Comportamento: depositar
    public void depositar(double valor) {
        if (valor > 0 && ativa) {
            saldo += valor;
            System.out.println("Depósito de R$"
                + valor + " realizado.");
        }
    }

    // Comportamento: consultar saldo
    public double consultarSaldo() {
        return saldo;
    }

    // Comportamento: verificar conta
    public boolean estaAtiva() {
        return ativa;
    }
}
```

Atributos + Métodos

> **Atributos guardam o estado**
> `saldo`, `ativa`, `titular`

> **Métodos operam sobre o estado**
> `depositar()` altera `saldo`

> **`this` refere-se ao objeto atual**
> Diferencia atributo do parâmetro de mesmo nome

🔑 Métodos definem o que um objeto sabe fazer. Atributos definem o que ele sabe sobre si mesmo.

Criação de Objetos — O Papel do `new`

```
// 1. Declaração da referência
ContaDigital conta;

// 2. Criação do objeto com new
conta = new ContaDigital();

// Ou em uma linha:
ContaDigital conta2 = new ContaDigital();

// 3. Acesso a atributos e métodos
conta2.titular = "Lucas";
conta2.saldo   = 500.0;
conta2.ativa   = true;

// 4. Chamada de método
conta2.depositar(200.0);

// 5. Resultado: saldo agora é 700.0
System.out.println(conta2.consultarSaldo());
```

O que o

Construtores — Inicializando Objetos

```
public class Transacao {

   String tipo;
   double valor;
   String descricao;

   // Construtor padrão
   public Transacao() {
      this.tipo = "INDEFINIDO";
      this.valor = 0.0;
   }

   // Construtor com parâmetros
   public Transacao(String tipo,
                    double valor,
                    String descricao) {
      this.tipo = tipo;
      this.valor = valor;
      this.descricao = descricao;
   }
}

// Uso:
Transacao t1 = new Transacao();
Transacao t2 = new Transacao("PIX", 150.0,
                             "Aluguel");
```

O que é um Construtor?

Mesmo nome da classe

`Transacao()` — começa com maiúscula

Sem tipo de retorno

Nem `void` — apenas o nome e os parâmetros

Chamado automaticamente pelo `new`

Garante estado inicial válido para o objeto

Sobrecarga é permitida

Diferentes assinaturas, mesmo nome

Construtor na Prática — Passo a Passo

```
public class Produto {
   String nome;
   double preco;
   int estoque;

   public Produto(String nome,
                  double preco,
                  int estoque) {
      this.nome = nome;
      this.preco = preco;
      this.estoque = estoque;
   }

   public void exibir() {
      System.out.println(nome + " - R$"
         + preco + " [" + estoque + " un]");
   }
}

// No main:
Produto p = new Produto("Fone BT", 299.90, 15);
p.exibir();
```

Rastreando a criação:

01

Declaração da referência

`Produto p` — variável criada, ainda sem objeto

02

Criação do objeto

`new Produto(...)` — memória alocada

03

Chamada do construtor

Parâmetros atribuídos a `this.nome`, `this.preco`, `this.estoque`

04

Referência atribuída

`p` aponta para o objeto criado

05

Saída: `Fone BT - R$299.9 [15 un]`

Método chamado sobre o objeto

Rastreando o Estado de um Objeto

Qual é o estado final do objeto?

```
public class CarrinhoDeCompras {
    String usuario;
    int itens;
    double total;

    public CarrinhoDeCompras(String usuario) {
        this.usuario = usuario;
        this.itens = 0;
        this.total = 0.0;
    }

    public void adicionar(double preco) {
        this.itens++;
        this.total += preco;
    }

    public void remover(double preco) {
        if (this.itens > 0) {
            this.itens--;
            this.total -= preco;
        }
    }
}

// Execução:
CarrinhoDeCompras c = new CarrinhoDeCompras("ana");
c.adicionar(89.90);
c.adicionar(45.00);
c.remover(89.90);
c.adicionar(120.00);
```

Rastreamento linha a linha:

Operação	itens	total
Construtor	0	0.0
adicionar(89.90)	1	89.90
adicionar(45.00)	2	134.90
remover(89.90)	1	45.00
adicionar(120.00)	2	165.00

✅ **Estado final:** itens=2, total=165.00

Encapsulamento

O problema sem encapsulamento:

```
public class CarteiraDigital {
    double saldo; // público — qualquer um altera!
}

// Isso seria possível (e perigoso!):
CarteiraDigital c = new CarteiraDigital();
c.saldo = -9999.99; // viola regra de negócio!
```

Com encapsulamento:

```
public class CarteiraDigital {
    private double saldo; // protegido!

    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
        }
    }
    public double getSaldo() {
        return saldo;
    }
}
```

Por que encapsular?

Proteger o estado
Impede valores inválidos nos atributos

Controlar o acesso
Define quem pode ler e quem pode alterar

Preservar regras de negócio
A validação fica centralizada no método

Facilitar manutenção
Mudança interna sem impacto externo

Modificadores de Acesso

Modificador	Mesma Classe	Mesmo Pacote	Subclasse	Qualquer lugar
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌
(padrão)	✅	✅	❌	❌
`private`	✅	❌	❌	❌

Na prática para encapsulamento:

Atributos → `private`

Nenhuma classe externa acessa diretamente

Métodos públicos → `public`

Interface de uso da classe para o mundo externo

Herança → `protected`

Acessível por subclasses, fechado para outros

Getters e Setters

```java
public class PerfilUsuario {

    private String nomeDeUsuario;
    private int nivel;
    private double xpAcumulado;

    // GETTER — leitura controlada
    public String getNomeDeUsuario() {
        return nomeDeUsuario;
    }

    public int getNivel() {
        return nivel;
    }

    // SETTER — escrita controlada (com validação)
    public void setNomeDeUsuario(String nome) {
        if (nome != null && nome.length() >= 3) {
            this.nomeDeUsuario = nome;
        }
    }

    public void adicionarXP(double xp) {
        if (xp > 0) {
            this.xpAcumulado += xp;
            this.nivel = (int)(this.xpAcumulado / 100);
        }
    }
}
```

Convenção em Java

Getter: `get` + NomeAtributo

Para `boolean`: usa-se `is` + NomeAtributo

Setter: `set` + NomeAtributo

Recebe o novo valor por parâmetro

Validação no setter

Garante que apenas valores válidos sejam aceitos

💡 Não é obrigatório ter getter e setter para todo atributo — exponha apenas o que for necessário!

Encapsulamento e Regra de Negócio

Cenário: carteira digital com limite de saque

```
public class CarteiraDigital {

    private double saldo;
    private double limiteSaqueDiario;
    private double saqueDiaAtual;

    public CarteiraDigital(double saldoInicial,
                           double limite) {
        this.saldo = saldoInicial;
        this.limiteSaqueDiario = limite;
        this.saqueDiaAtual = 0.0;
    }

    public boolean sacar(double valor) {
        double novoSaque = saqueDiaAtual + valor;
        if (valor <= 0) return false;
        if (valor > saldo) return false;
        if (novoSaque > limiteSaqueDiario) return false;

        saldo -= valor;
        saqueDiaAtual += valor;
        return true;
    }

    public double getSaldo() { return saldo; }
}
```

Regras protegidas:

Valor deve ser positivo
Validado dentro do método `sacar()`

Saldo não pode ser negativo
Impedido pela condição interna

Limite diário é respeitado
Regra de negócio centralizada no objeto

🔐 Nenhum código externo pode violar essas regras — elas estão *dentro* do objeto.

Herança — Reutilizando e Especializando

Conceito

Uma plataforma de streaming possui diferentes tipos de usuário — todos compartilham características comuns, mas cada tipo tem particularidades.



Benefícios da Herança

- [ ] **Reutilização de código**

  Atributos e métodos da superclasse ficam disponíveis nas subclasses

- [ ] **Especialização**

  Subclasses adicionam atributos e comportamentos específicos

- [ ] **Organização hierárquica**

  Relação "É UM": `UsuarioPremium` *é um* `Usuario`

⚠️ Herança modela relação "É UM". Se a relação for "TEM UM", use composição — não herança.

Especialização — Subclasse enriquece a Superclasse

O que a subclasse herda?

Herda (automático)

Atributos `public` e `protected`
Métodos `public` e `protected`

Não herda diretamente

Atributos e métodos `private`
Construtores da superclasse

Pode adicionar

Novos atributos e métodos específicos da subclasse

Pode sobrescrever

Redefinir métodos herdados com `@Override`

Chamando o construtor da superclasse

```
public class Usuario {
 protected String nome;
 protected String email;

 public Usuario(String nome, String email) {
 this.nome = nome;
 this.email = email;
 }

 public void login() {
 System.out.println(nome + " fez login.");
 }
}

public class UsuarioPremium extends Usuario {
 private String dataRenovacao;

 public UsuarioPremium(String nome,
 String email,
 String dataRenovacao) {
 super(nome, email); // chama construtor pai
 this.dataRenovacao = dataRenovacao;
 }
}
```

Herança em Código — Análise Completa

```
public class Dispositivo {
    protected String modelo;
    protected boolean conectado;

    public Dispositivo(String modelo) {
        this.modelo = modelo;
        this.conectado = false;
    }

    public void conectar() {
        this.conectado = true;
        System.out.println(modelo + " conectado.");
    }

    public String status() {
        return conectado ? "online" : "offline";
    }
}

public class Smartwatch extends Dispositivo {
    private int bateriaPercent;

    public Smartwatch(String modelo, int bateria) {
        super(modelo);
        this.bateriaPercent = bateria;
    }

    public void monitorarSaude() {
        System.out.println("Monitorando: " + modelo);
    }
}

// No main:
Smartwatch sw = new Smartwatch("Galaxy Watch", 80);
sw.conectar();       // herdado de Dispositivo
sw.monitorarSaude();  // próprio de Smartwatch
System.out.println(sw.status()); // herdado
```

O que acontece?

01

new Smartwatch("Galaxy Watch", 80)

Chama super(modelo) → conectado = false, modelo = "Galaxy Watch", depois bateriaPercent = 80

02

sw.conectar()

Método herdado: conectado = true → "Galaxy Watch conectado."

03

sw.monitorarSaude()

Método próprio: "Monitorando: Galaxy Watch"

04

sw.status()

Herdado. conectado é true → retorna "online"

Polimorfismo — Mesma Interface, Comportamentos Diferentes

Ideia Central

> **Mesma chamada de método → resultados diferentes dependendo do objeto real**

Em uma plataforma de pagamentos, diferentes meios de pagamento processam uma transação de formas distintas — mas todos respondem à mesma mensagem: `processar()`



Sobrescrita de Método (

Polimorfismo em Java — Código e Análise

```java
public class MeioPagamento {
    public void processar(double valor) {
        System.out.println("Processando R$" + valor);
    }
}

public class PagamentoPix extends MeioPagamento {
    @Override
    public void processar(double valor) {
        System.out.println("PIX: R$" + valor);
    }
}

public class PagamentoBoleto extends MeioPagamento {
    @Override
    public void processar(double valor) {
        System.out.println("Boleto: R$" + valor);
    }
}

// No main:
MeioPagamento p1 = new PagamentoPix();
MeioPagamento p2 = new PagamentoBoleto();
MeioPagamento p3 = new MeioPagamento();

p1.processar(200.0); // ?
p2.processar(350.0); // ?
p3.processar(50.0);  // ?
```

O que é impresso?

Chamada	Tipo real	Saída
p1.processar(200)	PagamentoPix	PIX: R$200.0
p2.processar(350)	PagamentoBoleto	Boleto: R$350.0
p3.processar(50)	MeioPagamento	Processando R$50.0

🔑 O tipo da referência (MeioPagamento) é o mesmo para todos — o comportamento depende do **objeto real** criado com new.

Como Raciocinar sobre Polimorfismo

Ao ver uma chamada de método com herança, faça estas perguntas nesta ordem:



Qual é o tipo da referência?

Tipo declarado à esquerda da variável



Qual objeto foi criado com `new`?

Tipo real do objeto em memória



Existe herança e sobrescrita?

A subclasse tem `@Override` no método?



Qual método está sendo chamado?

Nome e tipo do método invocado



Qual comportamento será executado?

Se há `@Override`, executa da subclasse. Se não, da superclasse.

O que é uma Classe Abstrata?

Conceito

Uma classe abstrata é um **modelo incompleto** — ela define a estrutura comum, mas deixa parte da implementação para as subclasses concretas.

Não pode ser instanciada

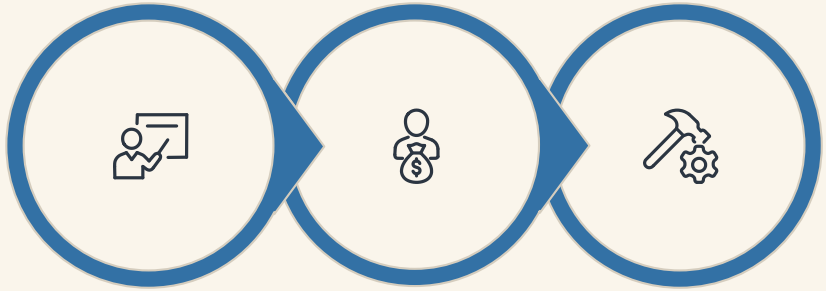
`new ClasseAbstrata()` → erro de compilação!

Pode ter métodos abstratos

Métodos sem corpo que subclasses são obrigadas a implementar

Pode ter métodos concretos

Implementações compartilhadas por todas as subclasses



Classe Abstrata | Herança | Implementação

💡 Pense em uma classe abstrata como um *contrato parcial*: diz o QUÊ deve existir, sem dizer COMO funciona em cada caso.

Classe Abstrata em Código

```java
// Classe abstrata
public abstract class Notificacao {
    protected String destinatario;
    protected String mensagem;

    public Notificacao(String destinatario,
                       String mensagem) {
        this.destinatario = destinatario;
        this.mensagem = mensagem;
    }

    // Método abstrato — sem corpo
    public abstract void enviar();

    // Método concreto — compartilhado
    public void registrarLog() {
        System.out.println("Log: msg para "
                           + destinatario);
    }
}

// Subclasse concreta
public class NotificacaoEmail extends Notificacao {
    public NotificacaoEmail(String dest, String msg) {
        super(dest, msg);
    }

    @Override
    public void enviar() {
        System.out.println("E-mail para: "
            + destinatario + " | " + mensagem);
    }
}

// Subclasse concreta
public class NotificacaoSMS extends Notificacao {
    public NotificacaoSMS(String dest, String msg) {
        super(dest, msg);
    }

    @Override
    public void enviar() {
        System.out.println("SMS: " + mensagem);
    }
}
```

Pontos-Chave

`abstract` na classe
Palavra-chave obrigatória antes de `class`

`abstract` no método
Sem chaves `{}` — apenas assinatura seguida de `;`

Subclasse concreta obrigada a implementar
Todos os métodos abstratos devem ter `@Override`

Polimorfismo funciona
`Notificacao n = new NotificacaoEmail(...);`

Métodos Abstratos — O Compromisso das Subclasses

```
public abstract class RelatorioFinanceiro {
    protected String periodo;
    protected double totalTransacoes;

    public RelatorioFinanceiro(String periodo) {
        this.periodo = periodo;
        this.totalTransacoes = 0.0;
    }

    // Método abstrato: cada tipo implementa
    public abstract void gerarRelatorio();

    // Método concreto: comportamento comum
    public void exibirCabecalho() {
        System.out.println("=== Relatório: "
                + periodo + " ===");
    }
}

public class RelatorioMensal
        extends RelatorioFinanceiro {

    public RelatorioMensal(String periodo) {
        super(periodo);
    }

    @Override
    public void gerarRelatorio() {
        exibirCabecalho(); // usa método herdado
        System.out.println("Total mensal: R$"
                + totalTransacoes);
    }
}
```

Regras dos Métodos Abstratos

- **Só existem em classes abstratas**
  Um método `abstract` exige que a classe também seja `abstract`
- **Subclasse concreta deve implementar todos**
  Se não implementar, ela também vira abstrata
- **Não têm corpo na classe abstrata**
  `public abstract void gerarRelatorio();` — ponto e vírgula, sem `{}`

Integração — Todos os Pilares Juntos

```java
// Classe abstrata
public abstract class Servico {
    protected String nome;
    protected boolean disponivel;

    public Servico(String nome) {
        this.nome = nome;
        this.disponivel = true;
    }

    public abstract double calcularCusto(int unidades);

    public void exibirStatus() {
        System.out.println(nome + ": "
            + (disponivel ? "disponível" : "indisponível"));
    }
}

// Herança + sobrescrita
public class ServicoNuvem extends Servico {
    private double precoPorGB;

    public ServicoNuvem(String nome, double precoPorGB) {
        super(nome); // construtor
        this.precoPorGB = precoPorGB; // encapsulamento
    }

    @Override
    public double calcularCusto(int gb) {
        return gb * precoPorGB;
    }
}

// Polimorfismo
Servico s = new ServicoNuvem("Storage Pro", 0.15);
s.exibirStatus();           // herdado
double custo = s.calcularCusto(500); // polimorfismo
```

Identificando os pilares:

Classe abstrata
`Servico` — não pode ser instanciada diretamente

Herança
`ServicoNuvem extends Servico`

Encapsulamento
`private double precoPorGB` — acesso controlado

Construtor
`super(nome)` encadeia ao construtor pai

Polimorfismo
Ref. `Servico` → objeto `ServicoNuvem`

Como Ler um Código Sem se Perder

Método de análise: siga esta sequência antes de olhar as alternativas.

Leia o contexto do enunciado

Entenda o problema antes de mergulhar no código

Identifique as classes

Quantas há? Qual a hierarquia? Alguma é abstrata?

Identifique atributos e métodos

O que cada classe armazena? O que ela faz?

Identifique os objetos criados

Quais objetos são criados no `main`? Com quais valores?

Observe as relações entre classes

Herança? Composição? Polimorfismo?

Acompanhe a execução linha a linha

Rastreie os valores dos atributos a cada chamada

Só então analise as alternativas

Elimine as incorretas com justificativa — não chute!

Questão Guiada — Resolução Passo a Passo

Contexto:

Um app de música gerencia playlists de usuários. Analise o código e determine a saída:

```
public class Playlist {
    private String nome;
    private int totalMusicas;

    public Playlist(String nome) {
        this.nome = nome;
        this.totalMusicas = 0;
    }

    public void adicionar(int quantidade) {
        if (quantidade > 0)
            this.totalMusicas += quantidade;
    }

    public String resumo() {
        return nome + ": " + totalMusicas + " músicas";
    }
}

// Main:
Playlist p = new Playlist("Foco Total");
p.adicionar(5);
p.adicionar(-2);
p.adicionar(3);
System.out.println(p.resumo());
```

Qual a saída do programa?

A) Foco Total: 6 músicas B) Foco Total: 8 músicas
C) Foco Total: 10 músicas D) Foco Total: 0 músicas
E) Erro de compilação

Raciocínio:

1. Construtor: nome="Foco Total", total=0
   Estado inicial após o new
2. adicionar(5): 5 > 0 → total=5
   Condição verdadeira — soma
3. adicionar(-2): -2 > 0 → **falso**
   Condição falsa — nada acontece
4. adicionar(3): 3 > 0 → total=8
   Condição verdadeira — soma
5. Saída: Foco Total: 8 músicas
   ✅ Resposta correta: **B**

Pegadinhas Conceituais — Não Confunda!

Classe × Objeto

Classe é o *modelo*. Objeto é a *instância* em memória.
`ContaDigital` = classe · `new ContaDigital()` = objeto

Atributo × Método

Atributo = variável de instância (estado).
Método = função que define comportamento.
Atributo *não* tem parênteses. Método *tem*.

Método × Construtor

Construtor: mesmo nome da classe, sem tipo de retorno, chamado pelo `new`.
Método: tem tipo de retorno (mesmo que `void`).

Referência × Objeto

`Conta c` = referência (variável que aponta).
`new Conta()` = o objeto real na heap.

Sobrecarga × Sobrescrita

Sobrecarga: mesmo nome, parâmetros diferentes, *na mesma classe*.
Sobrescrita: redefine método herdado, *na subclasse*, com `@Override`.

Classe Abstrata × Classe Concreta

Abstrata: tem `abstract`, **não** pode ser instanciada com `new`.
Concreta: pode ser instanciada diretamente.

Checklist de Preparação para o 1º GQ

Marque mentalmente cada item. Se algum estiver em dúvida, revise o conceito!

- [ ] Consigo trabalhar com tipos e variáveis em Java?
- [ ] Consigo interpretar operadores aritméticos, relacionais e lógicos?
- [ ] Consigo analisar estruturas condicionais (`if`, `else if`, `else`)?
- [ ] Consigo acompanhar a execução de loops (`for`, `while`, `do-while`)?
- [ ] Compreendo a declaração e chamada de métodos com parâmetros e retorno?
- [ ] Consigo trabalhar com métodos de String como `.length()`, `.equals()`, `.substring()`?
- [ ] Compreendo Arrays: declaração, índices e percorrimento?
- [ ] Compreendo ArrayList: `add`, `get`, `remove`, `size`?
- [ ] Sei diferenciar classe e objeto?
- [ ] Sei identificar atributos e métodos em uma classe?
- [ ] Compreendo como construtores funcionam e como são chamados?
- [ ] Compreendo encapsulamento: atributos `private` + métodos públicos?
- [ ] Sei interpretar modificadores de acesso (`public`, `private`, `protected`)?
- [ ] Compreendo herança: `extends`, `super()`, o que é herdado?
- [ ] Compreendo polimorfismo: referência da superclasse, objeto da subclasse?
- [ ] Sei identificar sobrescrita de método com `@Override`?
- [ ] Compreendo classes abstratas: `abstract`, não instanciável, métodos abstratos?
- [ ] Consigo acompanhar a execução de um programa Java com múltiplas classes?
- [ ] Consigo rastrear o estado de um objeto ao longo de chamadas de métodos?

🎯 Se você marcaria todos como ✅, está pronto para o 1º GQ! Boa prova!

SIMULADO FINAL · QUESTÃO 1 DE 10

Questão 1 — Fundamentos

Em um app de logística, um desenvolvedor escreveu o código abaixo. Qual o valor final de `pacotes`?

```
int pacotes = 10;
pacotes += 3;
pacotes *= 2;
System.out.println(pacotes);
```

- a) 16
- b) 20
- c) 26
- d) 23
- e) 13

GABARITO · QUESTÃO 1 DE 10

Questão 1 — Fundamentos

Em um app de logística, um desenvolvedor escreveu o código abaixo. Qual o valor final de `pacotes`?

```
int pacotes = 10;
pacotes += 3;
pacotes *= 2;
System.out.println(pacotes);
```

- a) 16
- b) 20
- ✅ c) **26** ← CORRETA
- d) 23
- e) 13

`pacotes = 10` → `+= 3` → `13` → `*= 2` → `26`. Operadores de atribuição composta executam na ordem em que aparecem no código.

SIMULADO FINAL · QUESTÃO 2 DE 10

Questão 2 — Condicional

Um sistema de assinatura verifica os dias restantes de um plano. O que é impresso pelo código abaixo?

```
int diasRestantes = 0;
if (diasRestantes > 7) {
    System.out.println("OK");
} else if (diasRestantes > 0) {
    System.out.println("Expirando");
} else {
    System.out.println("Expirado");
}
```

Qual a saída do programa?

- a) OK
- b) Expirando
- c) Expirado
- d) Nada é impresso
- e) Erro de compilação

GABARITO · QUESTÃO 2 DE 10

Questão 2 — Condicional

Um sistema de assinatura verifica os dias restantes de um plano. O que é impresso pelo código abaixo?

```
int diasRestantes = 0;
if (diasRestantes > 7) {
    System.out.println("OK");
} else if (diasRestantes > 0) {
    System.out.println("Expirando");
} else {
    System.out.println("Expirado");
}
```

Qual a saída do programa?

- a) OK
- b) Expirando
- ✅ c) **Expirado** ← CORRETA
- d) Nada é impresso
- e) Erro de compilação

`0 > 7` → falso. `0 > 0` → falso. Cai no `else`: imprime "Expirado". A ordem das condições importa — sempre avaliadas de cima para baixo.

SIMULADO FINAL · QUESTÃO 3 DE 10

Questão 3 — Loop

Analise o código abaixo. Qual é a saída impressa?

```
int soma = 0;
for (int i = 0; i < 4; i++) {
    soma += i;
}
System.out.println(soma);
```

- a) 10
- b) 4
- c) 6
- d) 0
- e) 3

GABARITO · QUESTÃO 3 DE 10

Questão 3 — Loop

Analise o código abaixo. Qual é a saída impressa?

```
int soma = 0;
for (int i = 0; i < 4; i++) {
    soma += i;
}
System.out.println(soma);
```

- a) 10
- b) 4
- ✅ c) **6** ← CORRETA
- d) 0
- e) 3

i assume os valores 0, 1, 2, 3. Soma: 0+1+2+3 = 6. Atenção: a condição é `i < 4` — quando `i=4` o loop para. O valor 4 não é somado!

SIMULADO FINAL · QUESTÃO 4 DE 10

Questão 4 — Strings

Um app de delivery usa o código abaixo. Qual é a saída impressa?

```
String pedido = "DEL-2024-XYZ";
System.out.println(pedido.substring(0, 3) + "-" + pedido.length());
```

Qual a saída do programa?

- a) DEL-12
- b) DEL-13
- c) DEL-2024
- d) DEL-9
- e) Erro de execução

GABARITO · QUESTÃO 4 DE 10

Questão 4 — Strings

Um app de delivery usa o código abaixo. Qual é a saída impressa?

```
String pedido = "DEL-2024-XYZ";
System.out.println(pedido.substring(0, 3) + "-" + pedido.length());
```

- a) DEL-12
- ✅ b) **DEL-13** ← CORRETA
- c) DEL-2024
- d) DEL-9
- e) Erro de execução

substring(0,3) retorna os caracteres nos índices 0, 1 e 2 → "DEL". "DEL-2024-XYZ".length() conta todos os caracteres incluindo hífens → 13. Resultado: "DEL-13".

SIMULADO FINAL · QUESTÃO 5 DE 10

Questão 5 — ArrayList

Um marketplace usa o código abaixo. Qual é a saída de `produtos.size()`?

```
ArrayList<String> produtos = new ArrayList<>();
produtos.add("Notebook");
produtos.add("Mouse");
produtos.add("Teclado");
produtos.remove("Mouse");
System.out.println(produtos.size());
```

- a) 3
- b) 2
- c) 1
- d) 0
- e) Erro de compilação

GABARITO · QUESTÃO 5 DE 10

Questão 5 — ArrayList

Um marketplace usa o código abaixo. Qual é a saída de `produtos.size()`?

```
ArrayList<String> produtos = new ArrayList<>();
produtos.add("Notebook");
produtos.add("Mouse");
produtos.add("Teclado");
produtos.remove("Mouse");
System.out.println(produtos.size());
```

- a) 3
- ✅ b) **2** ← CORRETA
- c) 1
- d) 0
- e) Erro de compilação

Foram adicionados 3 elementos. `remove("Mouse")` remove pelo valor (não pelo índice). Restam "Notebook" e "Teclado": `size()` = 2.

SIMULADO FINAL · QUESTÃO 6 DE 10

Questão 6 — Classe e Objeto

Sobre o trecho de código abaixo, qual afirmação é correta?

```
ContaCorrente cc1 = new ContaCorrente();
ContaCorrente cc2 = new ContaCorrente();
```

Qual afirmação é correta?

- a) `cc1` e `cc2` são classes
- b) `ContaCorrente` é um objeto
- c) `cc1` e `cc2` são objetos distintos na memória
- d) `cc1` e `cc2` compartilham os mesmos atributos
- e) `new` é desnecessário neste caso

GABARITO · QUESTÃO 6 DE 10

Questão 6 — Classe e Objeto

Sobre o trecho de código abaixo, qual afirmação é correta?

```
ContaCorrente cc1 = new ContaCorrente();
ContaCorrente cc2 = new ContaCorrente();
```

Qual afirmação é correta?

- a) cc1 e cc2 são classes
- b) ContaCorrente é um objeto
- ✅ c) **cc1 e cc2 são objetos distintos na memória** ← CORRETA
- d) cc1 e cc2 compartilham os mesmos atributos
- e) new é desnecessário neste caso

`cc1` e `cc2` são referências. Cada `new` cria um novo objeto na heap com atributos independentes. `ContaCorrente` é a classe (o modelo), não o objeto.

SIMULADO FINAL · QUESTÃO 7 DE 10

Questão 7 — Encapsulamento

Em uma classe Java, um atributo declarado como `private` pode ser acessado diretamente:

- a) Por qualquer classe do mesmo pacote
- b) Por qualquer subclasse
- c) Por qualquer classe do projeto
- d) Apenas pela própria classe onde foi declarado
- e) Por qualquer classe que importe o pacote

GABARITO · QUESTÃO 7 DE 10

Questão 7 — Encapsulamento

Em uma classe Java, um atributo declarado como `private` pode ser acessado diretamente:

- a) Por qualquer classe do mesmo pacote
- b) Por qualquer subclasse
- c) Por qualquer classe do projeto
- ✅ d) **Apenas pela própria classe onde foi declarado** ← CORRETA
- e) Por qualquer classe que importe o pacote

`private` é o modificador mais restritivo: acesso exclusivo da própria classe onde o atributo foi declarado. Nem subclasses acessam diretamente — elas devem usar getters e setters.

SIMULADO FINAL · QUESTÃO 8 DE 10

Questão 8 — Construtores

Sobre construtores em Java, qual afirmação é CORRETA?

- a) O construtor deve ter um tipo de retorno declarado (como void)
- b) O construtor é chamado manualmente após a criação do objeto
- c) Uma classe só pode ter um único construtor
- d) O construtor tem o mesmo nome da classe e é chamado pelo new
- e) Construtores não podem receber parâmetros

GABARITO · QUESTÃO 8 DE 10

Questão 8 — Construtores

Sobre construtores em Java, qual afirmação é CORRETA?

- a) O construtor deve ter um tipo de retorno declarado (como void)
- b) O construtor é chamado manualmente após a criação do objeto
- c) Uma classe só pode ter um único construtor
- ✅ d) **O construtor tem o mesmo nome da classe e é chamado pelo new** ← CORRETA
- e) Construtores não podem receber parâmetros

A: errado — construtor não tem tipo de retorno. B: errado — é chamado automaticamente pelo new. C: errado — sobrecarga permite múltiplos construtores. E: errado — construtores aceitam parâmetros normalmente.

SIMULADO FINAL · QUESTÃO 9 DE 10

Questão 9 — Herança e Polimorfismo

Analise o código abaixo. Qual é a saída impressa?

```
public class Sensor {
    public String coletar() { return "Sensor genérico"; }
}
public class SensorTemperatura extends Sensor {
    @Override
    public String coletar() { return "Temperatura: 36.5°C"; }
}
// No main:
Sensor s = new SensorTemperatura();
System.out.println(s.coletar());
```

- a) Sensor genérico
- b) Temperatura: 36.5°C
- c) Erro de compilação
- d) null
- e) Sensor genérico e Temperatura: 36.5°C

GABARITO · QUESTÃO 9 DE 10

Questão 9 — Herança e Polimorfismo

Analise o código abaixo. Qual é a saída impressa?

```
public class Sensor {
    public String coletar() { return "Sensor genérico"; }
}
public class SensorTemperatura extends Sensor {
    @Override
    public String coletar() { return "Temperatura: 36.5°C"; }
}
// No main:
Sensor s = new SensorTemperatura();
System.out.println(s.coletar());
```

- a) Sensor genérico
- ✅ b) **Temperatura: 36.5°C** ← CORRETA
- c) Erro de compilação
- d) null
- e) Sensor genérico e Temperatura: 36.5°C

Referência do tipo Sensor, mas o objeto real é SensorTemperatura. Como há @Override, o método da subclasse é executado — polimorfismo em ação. O Java sempre executa o método do objeto real, não do tipo da referência.

SIMULADO FINAL · QUESTÃO 10 DE 10

Questão 10 — Classes Abstratas

Sobre classes abstratas em Java, qual afirmação é CORRETA?

- a) Uma classe abstrata pode ser instanciada diretamente com new
- b) Um método abstrato pode ter corpo (implementação)
- c) Subclasses concretas podem ignorar métodos abstratos sem erro
- d) Subclasses concretas são obrigadas a implementar todos os métodos abstratos da superclasse
- e) Classes abstratas não podem ter construtores

GABARITO · QUESTÃO 10 DE 10

Questão 10 — Classes Abstratas

Sobre classes abstratas em Java, qual afirmação é CORRETA?

- a) Uma classe abstrata pode ser instanciada diretamente com new
- b) Um método abstrato pode ter corpo (implementação)
- c) Subclasses concretas podem ignorar métodos abstratos sem erro
- ✅ **d) Subclasses concretas são obrigadas a implementar todos os métodos abstratos da superclasse** ← CORRETA
- e) Classes abstratas não podem ter construtores

A: errado — classe abstrata não pode ser instanciada. B: errado — método abstrato não tem corpo. C: errado — se ignorar, a subclasse também vira abstrata (erro de compilação). E: errado — classes abstratas podem ter construtores, chamados via super().